

HuberLoss#

[https://pytorch.org/docs/stable/generated/torch.nn.modules.loss.HuberLoss.ht](https://pytorch.org/docs/stable/generated/torch.nn.modules.loss.HuberLoss.html)

Description:

Creates a criterion that uses a squared term if the absolute element-wise error falls below delta and a delta-scaled L1 term otherwise. This loss combines advantages of both L1Loss and MSELoss; the delta-scaled L1 region makes the loss less sensitive to outliers than MSELoss, while the L2 region provides smoothness over L1Loss near 0. See Huber loss for more information. For a batch of size NNN, the unreduced loss can be described as: If reduction is not none, then:

Function Signature

```
class torch.nn.modules.loss.HuberLoss(reduction='mean',  
delta=1.0)[source]#
```

Parameters

Shape:

```
forward(input, target)[source]#
```

Return type

Returns:

Tensor

Notes & Warnings

NOTE

Note When delta is set to 1, this loss is equivalent to SmoothL1Loss. In general, this loss differs from SmoothL1Loss by a factor of delta (AKA beta in Smooth L1). See SmoothL1Loss for additional discussion on the differences in behavior between the two losses.

Detailed Documentation

Documentation

Creates a criterion that uses a squared term if the absolute element-wise error falls below delta and a delta-scaled L1 term otherwise. This loss combines advantages of both L1Loss and MSELoss; the delta-scaled L1 region makes the loss less sensitive to outliers than MSELoss, while the L2 region provides smoothness over L1Loss near 0. See Huber loss for more information.

For a batch of size NNN, the unreduced loss can be described as:

If reduction is not none, then:

When delta is set to 1, this loss is equivalent to SmoothL1Loss. In general, this loss

differs from `SmoothL1Loss` by a factor of `delta` (AKA `beta` in `Smooth L1`). See `SmoothL1Loss` for additional discussion on the differences in behavior between the two losses.

`reduction` (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Default: 'mean'

`delta` (float, optional) – Specifies the threshold at which to change between delta-scaled L1 and L2 loss. The value must be positive. Default: 1.0

Input: $(*)^{(*)} (*)$ where $*^{**}$ means any number of dimensions.

Target: $(*)^{(*)} (*)$, same shape as the input.

Output: scalar. If reduction is 'none', then $(*)^{(*)} (*)$, same shape as the input.

Runs the forward pass.